

Entregable 1: Diagrama de clases UML con evidencia de comparación entre diseño propio y diseño sugerido por IA

Sistema de Gestión de Fincas e Inmuebles

Integrantes (Grupo 1)

- Fabio Perez Ruiz
- Maria Alejandra Ramos Naizir
- Karla Tom Cabarcas

<i>Elemento analizado</i>	<i>Diseño inicial del grupo</i>	<i>Sugerencia de la IA</i>	<i>Decisión final del grupo</i>	<i>Justificación</i>
Clase Inmueble	Clase base para representar propiedades	Convertirla en clase abstracta	Se convirtió a abstracta	Se identificó que Piso, Local y Edificio comparten atributos como dirección, descripción, código postal, precio de alquiler y disponibilidad. Por esta razón se decidió crear una clase abstracta Inmueble para evitar repetir información en varias clases y organizar mejor el modelo. Además, un inmueble genérico no tendría sentido como objeto independiente dentro del sistema.
Clase Alquiler	No existía inicialmente	Crear una clase independiente	Se agregó la clase alquiler	Inicialmente se pensó manejar el alquiler directamente desde Inmueble, pero esto hacía que la clase tuviera demasiadas responsabilidades. Se creó la clase Alquiler para representar la relación entre un Inquilino y un Inmueble, además de almacenar información propia del alquiler como fechas y estado.
Relación Edificio – Piso/Local	No estaba representada	Agregar relación entre clases	Se agregó la relación	El enunciado establece que un edificio puede contener pisos y locales. Por esta razón se agregó una relación entre Edificio y las clases Piso y Local, permitiendo representar la estructura real de los inmuebles administrados por la inmobiliaria.
Clase Factura	Se manejaban atributos separados (agua, gas, electricidad, etc.)	Representar cada factura como un objeto independiente	Se simplificaron los atributos para representar Factura como un objeto independiente	Se observó que una factura puede representar distintos conceptos y no únicamente servicios específicos. Por esta razón se decidió usar atributos generales como concepto, proveedor y costo, permitiendo registrar

				diferentes tipos de gastos sin necesidad de crear muchos atributos fijos.
Atributos particulares en Piso y Local	Las clases incluían atributos específicos como numeroPiso y tipoEspacio	Se sugirió evaluar si el atributo numeroPiso debía eliminarse de Local, y tipoEspacio de Piso para representar mejor las características específicas de cada tipo de inmueble	Se mantendrán ambos atributos en sus respectivas clases	Se decidió mantener los atributos numeroPiso y tipoEspacio como características particulares de las clases Piso y Local porque el enunciado establece que estos tipos de inmuebles pueden registrar información específica adicional a la heredada desde Inmueble. El atributo nPiso permite identificar la ubicación de un piso dentro de un edificio, mientras que tipoEspacio permite diferenciar la función de un local, como oficina, local comercial o bodega. Con esta decisión se buscó representar de manera más precisa las características propias de cada tipo de inmueble sin sobrecargar la clase general Inmueble.
Clase MovimientoBancario	Se manejaban categorías como atributos independientes (ingresosAlquiler, reformas, reparaciones, etc.)	Reemplazar categorías fijas por atributos generales como tipoMovimiento	Se modificó la estructura de MovimientoBancario	Se identificó que cada movimiento bancario debe representar una operación financiera individual dentro del sistema. Por esta razón se decidió eliminar atributos específicos para cada tipo de movimiento y reemplazarlos por atributos generales que permitan registrar distintos conceptos, como ingresos por alquiler, pagos de servicios, reformas o reparaciones, manteniendo un diseño más flexible y organizado.
Relación entre InmobiliariaAPP e Inquilino	La relación no estaba claramente definida	Mantener una colección de inquilinos administrada por la aplicación	Se agregó relación mediante agregación	El sistema necesita registrar, almacenar y consultar múltiples inquilinos. Por esta razón se decidió que la clase principal InmobiliariaAPP mantenga una colección de inquilinos para centralizar la administración de la información del sistema.
Relación entre Alquiler, Inquilino e Inmueble	La relación entre las clases estaba definida por agregación	Utilizar asociaciones en lugar de agregación	Se cambiaron las relaciones por asociaciones simples	Se decidió utilizar asociaciones entre Alquiler y las clases Inquilino e Inmueble debido a que el alquiler únicamente representa la relación entre

				ambas entidades dentro de un periodo determinado. Ni el inmueble ni el inquilino dependen de la existencia del alquiler para existir dentro del sistema.
Clase RepositorioDatos	No existía una clase para almacenar los datos y no perderlos	Crear una clase separada para manejar archivos	Se agregó la clase	El sistema requiere guardar y recuperar información de inmuebles, inquilinos y alquileres. Se decidió crear una clase independiente para organizar el manejo de archivos y evitar mezclar esta responsabilidad con la lógica principal de la aplicación.
Clase InmuebleServicio	Las operaciones de registro, consulta, modificación y eliminación se encontraban distribuidas entre distintas clases.	Centralizar la lógica de negocio en una clase de servicio independiente.	Se creó la clase InmuebleServicio o en reemplazo de InmobiliariaAPP	Se decidió concentrar las operaciones principales del sistema en una única clase encargada de la lógica de negocio, permitiendo que las entidades del modelo representaran únicamente la información y comportamiento propio de cada objeto.
Representación de enumeraciones	Los atributos con valores restringidos se manejaban como cadenas de texto.	Modelarlos mediante enumeraciones (<<enumeration >>) y establecer una relación de dependencia desde la clase que utiliza la enumeración hacia el tipo correspondiente	Se incorporaron las enumeraciones al UML	Se decidió representar las enumeraciones como elementos independientes del modelo y relacionarlas con las clases que las utilizan (Inquilino, MovimientoBancario y Factura). Esto permite expresar de forma explícita que dichos atributos solo pueden tomar un conjunto definido de valores, mejorando la legibilidad del UML y manteniendo la coherencia con la implementación en Java mediante enum.
Multiplicidades del UML	Varias relaciones utilizaban multiplicidades que no representaban todas las situaciones posibles del sistema.	Revisar las cardinalidades de cada asociación de acuerdo con las reglas del negocio	Se actualizaron las multiplicidades del modelo	Se corrigieron multiplicidades, por ejemplo entre Edificio y Piso/Local, permitiendo representar que un edificio puede existir sin tener inicialmente pisos o locales registrados (0..*), logrando un modelo más consistente con el comportamiento esperado del sistema.
Relaciones entre las clases del sistema	Algunas relaciones del diagrama	Revisar la representación de las relaciones	Se realizaron las correcciones propuestas.	Se ajustaron las relaciones del diagrama para que representaran adecuadamente

	presentaban inconsistencias en el tipo de relación, la dirección de las dependencias y la ubicación de la composición.	para que reflejaran correctamente la interacción entre las clases y las responsabilidades de cada una. Se sugirió ubicar correctamente la composición entre VentanaPrincipal y los paneles, invertir la dependencia entre Inmueble e InmuebleServicio, corregir la dirección de la relación entre Inquilino e InmuebleServicio y reemplazar las composiciones entre Factura/MovimientoBancario e Inmueble por asociaciones, ya que estas entidades solo almacenan el identificador del inmueble		la dependencia y el ciclo de vida de los objetos. La composición quedó ubicada en VentanaPrincipal, las dependencias con InmuebleServicio reflejan correctamente que la capa de servicios utiliza las clases del modelo, y las relaciones de Factura y MovimientoBancario con Inmueble se representaron como asociaciones, puesto que estas entidades únicamente conservan una referencia mediante el identificador del inmueble y no una instancia del objeto.
Clases de la interfaz gráfica	Existían dudas sobre si el UML debía incluir las clases base de Swing utilizadas por la interfaz.	Incorporar las clases JFrame, JPanel y JDialog para reflejar la jerarquía completa de la interfaz gráfica.	No se incorporaron las clases de Swing al UML	Se decidió representar únicamente las clases desarrolladas por el grupo (VentanaPrincipal, paneles y diálogos propios), ya que las clases de Swing pertenecen a la biblioteca de Java y representan detalles de implementación que no aportan al modelo de clases del sistema
Métodos representados en el UML	Se contemplaba representar métodos públicos y privados en las clases.	Mostrar únicamente los métodos públicos, ya que los privados corresponden a detalles internos	Se representaron únicamente los métodos públicos.	Se decidió mantener el diagrama enfocado en el comportamiento visible de las clases, evitando incluir métodos auxiliares que no aportan al entendimiento del diseño del sistema.

		de implementación .		
Representación de paquetes en el UML	El diagrama mostraba todas las clases en un único espacio, sin agruparlas por paquetes.	Organizar las clases dentro de paquetes (modelo, vista, servicio, persistencia, util, etc.) para representar la arquitectura completa del proyecto.	No se representaron los paquetes en el UML.	El grupo decidió mantener el diagrama centrado en las clases y sus relaciones, ya que el objetivo del entregable era modelar la estructura orientada a objetos del sistema y no la organización física del código. La representación de paquetes se dejó para la implementación del proyecto en Java.